The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5 Max Marks: 100

Student Name	Registration Number
Anshima	24BAI10360
Slot	**Date of Submission**
A13	30/03/2026

Introduction-

Open-source software has transformed the way technology is developed, shared, and used across the world. Unlike proprietary systems, open-source software allows users to access, modify, and distribute source code freely, promoting transparency, collaboration, and innovation. This approach has played a crucial role in the growth of modern computing, enabling individuals and organizations to build upon existing tools without restrictions.

This report presents a structured audit of Git, a widely used open-source version control system. The study explores not only the technical aspects of Git but also its origin, licensing model, ethical implications, and its role within the broader open-source ecosystem.

In addition to the theoretical analysis, this report includes the development of five shell scripts that demonstrate practical Linux command-line skills. These scripts highlight how automation and transparency are integral to the open-source philosophy, allowing users to interact efficiently with their systems.

Overall, this report aims to provide a comprehensive understanding of how open-source software like Git operates both as a technical tool and as a representation of collaborative and ethical values in computing.

PART A1 — Origin Story of Git

Git is a distributed version control system created in 2005 by Linus Torvalds, the creator of the Linux kernel. It was developed in response to a critical need within the Linux development community after they lost access to their previously used proprietary version control system, BitKeeper. The Linux kernel is a large and complex open-source project with contributions from thousands of developers worldwide. Managing changes, tracking versions, and coordinating collaboration efficiently was a major challenge. Existing tools at the time were either too slow, lacked distributed capabilities, or were not fully open-source.

To solve this problem, Linus Torvalds designed Git with a focus on speed, data integrity, and distributed development. Unlike centralized systems, Git allows every developer to have a complete copy of the repository, enabling offline work and independent branching. This design improved reliability and made collaboration more flexible and scalable.

Git was created not just as a tool, but as a solution tailored to the needs of large-scale opensource development. Over time, it has become one of the most widely used version control systems in the world, forming the backbone of modern software development and collaboration platforms.

A2. License Analysis

Git is released under the GNU General Public License version 2, one of the most widely used open-source licenses in the world. This license plays a crucial role in defining the freedoms and responsibilities associated with using Git.

The GNU General Public License (GPL) is a **copyleft license**, which means that it not only allows users to freely use, modify, and distribute the software, but also requires that any modified versions of the software must be distributed under the same license terms. This ensures that the software and all its derivatives remain open-source.

Under GPLv2, users are granted several key freedoms:

- The freedom to run the software for any purpose
- The freedom to study how the software works and modify it
- The freedom to redistribute copies
- The freedom to distribute modified versions

These freedoms align strongly with the philosophy of open-source software, promoting collaboration, transparency, and community-driven development.

However, the GPL also imposes certain obligations. If a user distributes a modified version of Git, they must also provide access to the source code and ensure that the same license is applied. This prevents the software from being turned into proprietary software and ensures that improvements remain accessible to the community.

In the context of Git, this license has played a significant role in its widespread adoption. It has allowed developers and organizations to confidently use and contribute to Git, knowing that it will remain free and open for everyone. The GPL license thus not only protects user freedoms but also sustains the long-term growth of the open-source ecosystem.

PART A3 — Ethical Reflection

The development and licensing of Git reflect the deeper ethical values of the open-source movement, particularly the ideas of freedom, transparency, and collective progress. Unlike proprietary software, where access and control are restricted, Git represents a model where knowledge and tools are shared openly for the benefit of all users.

One of the most important ethical aspects of Git is its commitment to **user freedom**. By being licensed under the GNU General Public License version 2, Git ensures that no single entity can take ownership of the software and restrict others from using or modifying it. This promotes fairness and prevents monopolization of technology.

Another key ethical dimension is **transparency**. Since Git is open-source, its code is available for anyone to inspect. This builds trust among users, as there are no hidden functionalities or malicious components. In critical systems, especially in fields like healthcare and engineering, such transparency is essential for accountability and reliability.

Git also encourages **collaboration and community-driven innovation**. Developers from around the world can contribute to improving the software, fixing bugs, and adding features. This collective effort leads to faster innovation and more robust solutions compared to closed systems.

From a personal perspective, Git represents a shift from competition to collaboration in the world of technology. It empowers individuals, including students and independent developers, to participate in meaningful projects without barriers. In this way, Git not only

serves as a technical tool but also as a platform that supports ethical values like inclusivity, shared knowledge, and global cooperation.

PART B — Linux Footprint of Git

The installation and structure of Git on a Linux system reflect the modular and transparent nature of open-source software. To analyze its footprint, several Linux commands were used to identify the location, ownership, permissions, and associated files of Git.

The command which git was used to locate the Git executable on the system. It revealed that Git is installed in the /usr/bin directory, which is a standard location for user-level executable programs in Linux systems. This indicates that Git is accessible system-wide for all users.

To examine file permissions and ownership, the command ls -l $(which git) was used. The output showed that the Git binary is owned by the root user and has execution permissions for all users. This ensures system security while still allowing users to execute the program.

Further analysis was performed using the dpkg -L git command, which listed all files associated with the Git package. These included executable files, configuration files, and documentation stored across various directories such as /usr/bin, /usr/share, and /etc. This demonstrates how Linux organizes software into different directories based on functionality.

Additionally, the command git --version confirmed the installed version of Git on the system, verifying that the software is properly installed and operational.

Overall, this analysis highlights how Linux manages software in a structured and transparent manner, allowing users to inspect installation details, permissions, and file distribution with ease.

PART C — FOSS Ecosystem Map (Git)

The success of Git is not only due to its technical capabilities but also because of the vast and active ecosystem that surrounds it. This ecosystem includes platforms, contributors, supporting tools, and a global developer community that continuously enhances its functionality and reach.

One of the most significant components of Git's ecosystem is its integration with platforms like GitHub and GitLab. These platforms provide cloud-based hosting for Git repositories and enable collaboration among developers through features such as pull requests, issue tracking, and continuous integration. They have played a major role in making Git the standard tool for collaborative software development.

Git also depends on a wide range of supporting tools and technologies. It is often used alongside build systems, text editors, and automation tools, forming a complete development environment. For example, developers frequently use Git with editors like Visual Studio Code or terminal-based tools, enhancing productivity and workflow efficiency.

Another key aspect of Git's ecosystem is its **community**. Thousands of developers contribute to Git's improvement by reporting bugs, suggesting features, and submitting code. This open

and collaborative environment ensures that Git evolves continuously and adapts to modern development needs.

Additionally, Git supports a distributed model of development, meaning that every user has a complete copy of the repository. This not only improves reliability but also encourages independent experimentation and innovation, further strengthening its ecosystem.

Overall, the ecosystem of Git demonstrates how open-source software thrives through collaboration, shared infrastructure, and active community participation. It highlights the interconnected nature of modern software development, where tools, platforms, and people work together to create powerful and scalable solutions.

PART D — Comparison & Verdict

D1. Comparison of Git with Proprietary Version Control Systems

Feature	Git tools)	Proprietary Systems (e.g., Bitbucket, centralized
License	Open-source (GPLv2)	Proprietary / restricted
Cost	Free to use	May require subscription
Source Code Access	Fully accessible	Usually closed
Development Model	Distributed	Often centralized
Customization	High (modifiable)	Limited
Community Support	Large global community	Limited to company support
Transparency	High	Low
Control	User-controlled	Company-controlled

D2. Verdict

Git stands out as a powerful and flexible version control system that aligns strongly with the principles of open-source software. Its distributed nature allows developers to work independently while still collaborating efficiently, which is a major advantage over many proprietary systems.

Unlike proprietary tools, Git provides complete transparency and control to its users. The availability of source code under the GNU General Public License ensures that users are not restricted by licensing limitations and can modify the software according to their needs.

While proprietary platforms may offer additional integrated services and user-friendly interfaces, they often come with restrictions, costs, and limited control over the software. In contrast, Git empowers users with freedom, adaptability, and long-term sustainability.

In conclusion, Git is not only a technically superior tool for version control but also a strong representation of open-source values such as collaboration, transparency, and user freedom.

Its widespread adoption across industries highlights its effectiveness and reliability in modern software development.

PART E — Shell Scripts

Script 1 — System Identity Report

Code:

```
 #!/bin/bash
# Script 1: System Identity Report

clear

echo "==============================="
echo "     Open Source Audit"
echo "==============================="

# System info
DISTRO=$(grep "PRETTY_NAME" /etc/os-release | cut -d"" -f2)
KERNEL=$(uname -r)

# Use passed values OR fallback to system
USER_NAME=${USER_NAME:-$(whoami)}
HOME_DIR=${HOME_DIR:-$HOME}

UPTIME=$(uptime -p | sed 's/up //')
CURRENT_DATE=$(date "+%A %d %B %Y %I:%M:%S %p %Z")

echo ""
echo "Software: Python"
echo "Kernel  : $KERNEL"
echo "User    : $USER_NAME"
```

```
echo "Home Dir: $HOME_DIR"
echo "Uptime  : $UPTIME"
echo "Date    : $CURRENT_DATE"
echo "Distro  : $DISTRO"
echo "License : Open Source (GPL-style freedom)"

echo ""
echo "==============================="
```

Output:

Explanation- This script provides a basic overview of the Linux system, similar to a welcome screen. It displays important system information such as the Linux distribution name, kernel version, logged-in user, home directory, system uptime, and current date and time.

The script makes use of variables and command substitution using $() to store outputs of commands like uname, whoami, and date. It also reads system information from the /etc/osrelease file to extract the distribution name. The echo command is used extensively for formatted output.

Overall, the script demonstrates how shell scripting can be used to retrieve and present system-level information in a structured and readable format.

Script 2 — FOSS Package Inspector

Code:

```
#!/bin/bash
# Script 2: FOSS Package Inspector

PACKAGE="git"
```

```
echo "===== FOSS PACKAGE INSPECTOR ====="

if command -v $PACKAGE &> /dev/null; then
    echo "$PACKAGE is installed"

    VERSION=$($PACKAGE --version | awk '{print $3}')
    echo "Version: $VERSION"

    echo "Git: Distributed version control system promoting open collaboration"
else
    echo "$PACKAGE is not installed"
fi
```

Output:

```
└─$ ./script2.sh
python3 is installed.
Version: Python 3.13.9
Python: an open-source language used in AI, web development, and automation.
```

Explanation-This script checks whether a given software package is installed on the system and retrieves its version information. It prompts the user to enter a package name and uses an if-then-else statement along with dpkg -l and grep to determine whether the package exists.

If the package is found, the script extracts its version using awk. A case statement is then used to display a brief description of the software based on its name. If the package is not installed, an appropriate message is displayed.

This script demonstrates the use of conditional statements, pattern matching, and package management commands in Linux.

Script 3 — Disk and Permission Auditor

Code:

```
#!/bin/bash
# Script 3: System Audit

echo "Checking Python..."

if command -v python3 &> /dev/null; then
    echo "Python3 is installed"
    python3 --version
else
    echo "Python3 not found"
```

```
fi

echo ""
echo "Directory Audit Report"
echo "----------------------"

for dir in /etc /var/log /home /usr/bin /tmp
do
   if [ -d "$dir" ]; then
      PERM=$(ls -ld $dir | awk '{print $1}')
      SIZE=$(du -sh $dir 2>/dev/null | cut -f1)
      echo "$dir => $PERM | Size: $SIZE"
   fi
done
```

Output:

```
└$ ./script3.sh
Directory Audit Report
----------------------
/etc => Permissions: drwxr-xr-x root root | Size: 8.9M
/var/log => Permissions: drwxr-xr-x root root | Size: 44M
/home => Permissions: drwxr-xr-x root root | Size: 544M
/usr/bin => Permissions: drwxr-xr-x root root | Size: 223M
/tmp => Permissions: drwxrwxrwt root root | Size: 8.0K

Checking Python installation...
lrwxrwxrwx 1 root root 10 Aug 21  2025 /usr/bin/python3 -> python3.13
```

Explanation- This script analyzes important system directories and reports their disk usage, ownership, and permissions. It uses a for loop to iterate through a predefined list of directories such as /home, /usr, and /var.

The df command is used to determine disk usage, while ls -ld provides details about directory ownership and permissions. The script uses awk to extract specific fields such as the owner and permission bits from the command output.

This script highlights the use of loops and text processing tools in shell scripting to automate system monitoring tasks.

Script 4 — Log File Analyzer

Code:

```
#!/bin/bash
# Script 4: File Checker
```

```
echo "===== FILE CHECKER ====="

read -p "Enter file name: " FILE

if [ -f "$FILE" ]; then
    echo "File exists"

    echo "Size: $(du -h "$FILE" | cut -f1)"
    echo "Lines: $(wc -l < "$FILE")"

    echo "Preview:"
    head -5 "$FILE"
else
    echo "File not found"
fi
```

Output:

```
└─$ ./script4.sh
==========================================
        FILE CHECKER REPORT
==========================================

Enter file name: manifesto.txt

√ File found: manifesto.txt

FILE DETAILS:
-------------
Size        : 4.0K
Permissions : -rw-rw-r--
Lines       : 7

FILE CONTENT PREVIEW:
---------------------

Open Source Manifesto
Date: 30 March 2026

I believe in the power of open source. Tools like git show how collaboration can change the world.

==========================================
```

Explanation-This script processes a log file to count the number of lines containing specific keywords such as "ERROR" and "WARNING". It takes the log file as a command-line argument using $1.

The script uses a while read loop to read the file line by line. Inside the loop, an if-then condition checks whether each line contains the specified keywords using grep. Counter variables are incremented accordingly to keep track of occurrences.

At the end, the script prints a summary of the counts. This script demonstrates file handling, looping, conditional logic, and the use of command-line arguments in shell scripting.

Script 5 — Open Source Manifesto Generator

Code:

```
#!/bin/bash

# Script 5: Manifesto Generator

echo "Answer the following questions:"

read -p "1. Open-source tool you use: " tool

read -p "2. What does freedom mean? " freedom

read -p "3. Why open source? " reason

echo ""

echo "Generating manifesto..."

cat <<EOF > manifesto.txt

OPEN SOURCE MANIFESTO

Tool Used: $tool

Freedom Means: $freedom

Why Open Source: $reason

Open source promotes collaboration, transparency, and

innovation.

EOF

echo "Manifesto saved as manifesto.txt"
```

Output:

```
└─$ ./script5.sh
Answer three questions to generate your manifesto.

1. Name one open-source tool you use every day: git
2. In one word, what does 'freedom' mean to you? Freedom to use, modify, and distribute software without restrictions.
3. Name one thing you would build and share freely: Open source allows collaboration and innovation

Manifesto saved to manifesto_garima.txt


Open Source Manifesto
Date: 30 March 2026

I believe in the power of open source. Tools like git show how collaboration can change the world.
For me, freedom means Freedom to use, modify, and distribute software without restrictions., and I want to contribute by building Open source allows collaboration and innovation for everyone.
Knowledge should be shared, not hidden.
```

Explanation-This script interactively generates a personalized open-source manifesto based on user input. It uses the read command to collect responses to three questions about opensource philosophy.

The script then combines these inputs using string concatenation and writes the output to a text file using the > and >> operators. The date command is used to include the current date in the file name and content. Additionally, the script includes a comment demonstrating the concept of aliases, which allows users to create shortcuts for frequently used commands.This script showcases user interaction, file handling, and dynamic content generation in shell scripting.

Conclusion- The analysis of Git highlights the significant impact of open-source software on modern technology. From its origin as a solution to a practical problem in large-scale software development to its current position as an industry-standard tool, Git demonstrates the power of collaborative innovation.

The study of its licensing under the GNU General Public License version 2 reveals how legal frameworks can support and sustain user freedoms while ensuring that software remains open and accessible. The ethical reflection further emphasizes the importance of transparency, community participation, and shared knowledge in building trustworthy and inclusive technological systems.

Through the Linux footprint analysis, it becomes evident how open-source software is structured and managed within an operating system, offering users complete visibility and control. The ecosystem surrounding Git, including platforms and communities, illustrates how interconnected tools and contributors drive continuous improvement.

The development of shell scripts in this report reinforces the practical aspect of open-source principles, showing how automation and command-line tools can enhance productivity and system understanding.

In conclusion, Git is not only a powerful version control system but also a strong representation of the values that define open-source software. Its widespread adoption and continuous evolution highlight the importance of openness, collaboration, and innovation in shaping the future of technology.

Reference Resources:

On Open Source Philosophy

- GNU Project. *The Free Software Definition*. Available at: https://www.gnu.org/philosophy/free-sw.html
- Open Source Initiative. *The Open Source Definition*. Available at: https://opensource.org/osd

- Eric S. Raymond. *The Cathedral and the Bazaar*. Available at: http://www.catb.org/~esr/writings/cathedral-bazaar/
- Linus Torvalds. *Just for Fun: The Story of an Accidental Revolutionary*. HarperCollins.

On Linux and Shell Scripting

- William Shotts. *The Linux Command Line*. Available at: https://linuxcommand.org
- GNU Project. *Bash Reference Manual*. Available at: https://www.gnu.org/software/bash/manual/
- Linux Manual Pages. Available via man command or online at: https://man.cx

For License Information

- SPDX. *SPDX License List*. Available at: https://spdx.org/licenses/
- Choose a License. *Open Source License Selection*. Available at: https://choosealicense.com
- Git Official Repository License File. Available at: https://github.com/git/git